EduGenie

Google Gemini Powered Learning Assistant

Project Coding Documentation

Presented by

Ranjini S

Department: 2nd BCA
Shift: 2

Academic Project

1. Project Overview

EduGenie is a learning assistant designed to help students understand academic topics through an AI-powered conversational interface. The application uses Google Gemini to generate explanations, summaries, examples, practice questions and study assistance based on the student's prompt.

2. Objectives

• Provide quick and understandable explanations of academic concepts.
• Generate summaries and examples for revision.
• Support question-and-answer based learning.
• Create a simple, student-friendly AI learning interface.
• Demonstrate the integration of a generative AI API into an educational application.

3. Technologies Used

Frontend: HTML, CSS, JavaScript
Backend/API integration: Python Flask
AI Service: Google Gemini API
Data format: JSON
Development tools: VS Code / Web Browser

4. Project Structure

edugenie/
■■■ app.py
■■■ requirements.txt
■■■ .env
■■■ templates/
■ ■■■ index.html
■■■ static/
■■■ style.css
■■■ script.js

5. Backend Code – app.py

The following sample Flask backend receives a student's question and sends it to the Gemini model.

```
import os
from flask import Flask, render_template, request, jsonify
from dotenv import load_dotenv
from google import genai

load_dotenv()

app = Flask(__name__)
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
    data = request.get_json()
```

```
    question = data.get("question", "").strip()

    if not question:
        return jsonify({"answer": "Please enter a question."})

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=(
            "You are EduGenie, a helpful learning assistant. "
            "Explain concepts clearly for a college student. "
            "Use simple language and examples when useful.\n\n"
            f"Student question: {question}"
        )
    )

    return jsonify({"answer": response.text})

if __name__ == "__main__":
    app.run(debug=True)
```

6. Frontend Code – index.html

A simple interface allows the student to enter a question and display the AI-generated response.

```
<!DOCTYPE html>
<html>
<head>
  <title>EduGenie</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <main class="container">
    <h1>EduGenie</h1>
    <p>Google Gemini Powered Learning Assistant</p>

    <textarea id="question"
      placeholder="Ask EduGenie a study question..."></textarea>
    <button onclick="askEduGenie()">Ask EduGenie</button>

    <section>
      <h2>Answer</h2>
      <div id="answer">Your answer will appear here.</div>
    </section>
  </main>
  <script src="/static/script.js"></script>
</body>
</html>
```

7. JavaScript – script.js

JavaScript sends the student's question to the Flask endpoint and updates the answer area.

```
async function askEduGenie() {
  const question = document.getElementById("question").value;
  const answer = document.getElementById("answer");

  if (!question.trim()) {
    answer.textContent = "Please enter a question.";
    return;
  }

  answer.textContent = "Thinking...";

  const response = await fetch("/ask", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({question})
  });

  const data = await response.json();
  answer.textContent = data.answer;
}
```

8. CSS – style.css

CSS can be used to create a clean, responsive student dashboard with a question box, button and answer panel.

```
body {
  font-family: Arial, sans-serif;
  background: #f4f7fb;
  margin: 0;
}

.container {
  max-width: 800px;
  margin: 60px auto;
  padding: 30px;
  background: white;
  border-radius: 12px;
}

textarea {
  width: 100%;
  min-height: 140px;
  padding: 12px;
  box-sizing: border-box;
}

button {
  margin-top: 12px;
  padding: 12px 20px;
  cursor: pointer;
}

#answer {
  margin-top: 10px;
  padding: 15px;
  background: #f1f1f1;
  white-space: pre-wrap;
}
```

9. requirements.txt

flask
google-genai
python-dotenv

```
flask
google-genai
python-dotenv
```

10. How to Run

1. Install Python 3.10+.
2. Create a virtual environment.
3. Install dependencies using **pip install -r requirements.txt**.
4. Create a Gemini API key through Google's developer platform and store it in the environment file.
5. Start the Flask application using **python app.py**.
6. Open the local URL shown by Flask in a web browser.

11. Expected Features

• Ask questions in natural language.
• Receive AI-generated explanations.
• Request summaries and examples.

• Use the assistant for revision and practice.
• Simple and responsive interface.

12. Future Enhancements

User login, subject-wise learning modes, quiz generation, progress tracking, conversation history, multilingual support, teacher dashboards and safer content controls can be added in future versions.

13. Conclusion

EduGenie demonstrates how generative AI can be integrated into an educational application to provide interactive learning support. The project combines a web interface, a Python backend and the Google Gemini API in a practical BCA project.